Spring XML-Based Configuration

1. Why XML Configuration Exists

In Spring, annotations are not the only way to tell the IoC container which objects it should manage.

Before annotation-based configuration became common, Spring applications were mostly configured using XML files.

XML configuration is another way of telling Spring:

> These are my classes.
>
> These are the objects you need to create.
>
> These are their dependencies.
>
> Please manage them inside the IoC container.

---

2. What Does Spring Actually Need?

The main job of the Spring IoC container is to:

> Create objects, connect their dependencies, manage their lifecycle, and destroy them when required.

To do this, Spring needs configuration information such as:

1. Which classes should become beans?
2. What should be the bean name?
3. Should the bean be `singleton` or `prototype`?
4. Which dependency should be injected?
5. Which bean should be preferred if multiple beans of the same type exist?
6. Which method should run after bean creation?

7. Which method should run before bean destruction?

This information is called **configuration metadata**.

3. Configuration Metadata in Spring

Spring can receive configuration metadata in different ways:

Configuration Style	Where Configuration Is Written
XML-based configuration	Inside an XML file
Annotation-based configuration	Inside Java classes
Java-based configuration	Inside Java config classes using `@Configuration` and `@Bean`

No matter which style we use, Spring converts this configuration metadata into internal objects called **BeanDefinition** objects.

```
Configuration Metadata
        |
        |---- XML
        |---- Annotations
        |
   BeanDefinition
        |
   IoC Container
        |
Creates + Wires + Manages Beans
```

The important idea is:

> XML, annotations, and Java config are only different ways of giving instructions to Spring.
>
> Internally, Spring uses those instructions to create bean definitions.

4. Why Should We Learn XML Configuration?

Even though modern Spring projects mostly use annotations and Java-based configuration, XML is still useful to understand.

Reason 1: Legacy Projects

Many old Spring applications still use XML configuration.

If you work on an older enterprise project, you may find files like:

```
beans.xml
applicationContext.xml
spring-context.xml
```

Understanding XML configuration helps you read and maintain such projects.

Reason 2: Better Understanding of Spring Internals

Annotations sometimes hide what Spring is doing internally.

XML makes the wiring more visible because we manually write:

- Which bean should be created
- What its name should be
- Which dependency should be injected
- Which lifecycle method should be called

This makes XML a good way to understand the foundation of Spring.

---

5. Recommended Project Structure

A simple XML-based Spring project can follow this structure:

```
src
 └── main
     ├── java
     │   └── in
     │       └── strikes
     │           ├── Main.java
     │           ├── PaymentService.java
     │           └── OrderService.java
     │
     └── resources
         └── beans.xml
```

The XML file should be placed inside:

```
src/main/resources
```

This is important because files inside `resources` become available on the **classpath**.

So when we write:

```
new ClassPathXmlApplicationContext("beans.xml");
```

Spring searches for `beans.xml` from the classpath.

Part 1: Creating the First XML Bean

6. Create a Simple Java Class

```
package in.strikes;

public class PaymentService {

    public PaymentService() {
        System.out.println("PaymentService object created");
    }

    public void pay() {
        System.out.println("Payment successful");
    }
}
```

This is a normal Java class.

There is no annotation like `@Component`.

So Spring will not automatically detect it.

We will register this class as a bean manually using XML.

---

7. Create `beans.xml`

Inside `src/main/resources`, create a file named:

```
beans.xml
```

Add the basic XML structure:

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

</beans>
```

For now, this XML file is empty.

The root `<beans>` tag is the place where we write all bean definitions.

---

8. Register the First Bean

Now add this inside the `<beans>` tag:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

Complete `beans.xml`:

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="paymentService" class="in.strikes.PaymentService"/>
```

```
</beans>
```

This line means:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

> Spring, create and manage one object of `PaymentService`.
>
> Keep its bean name as `paymentService`.

Here:

XML Attribute	Meaning
`id`	Name of the bean inside the Spring container
`class`	Fully qualified class name whose object Spring should create

9. XML Bean Compared with `@Component`

In annotation-based configuration, we write:

```
@Component
public class PaymentService {
}
```

Spring usually gives this bean the default name:

```
paymentService
```

In XML configuration, we manually write:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

So this XML configuration is conceptually similar to:

```
@Component("paymentService")
public class PaymentService {
}
```

The difference is:

Annotation Config	XML Config
Configuration is written inside Java class	Configuration is written inside XML file
Spring detects class using component scanning	Spring reads bean definition from XML
Bean name may be generated automatically	Bean name is usually written manually

10. Load XML Configuration in Java

Create `Main.java`:

```
package in.strikes;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Main {

    public static void main(String[] args) {

        ApplicationContext context =
                new ClassPathXmlApplicationContext("beans.xml");

        PaymentService paymentService =
                context.getBean("paymentService", PaymentService.class);

        paymentService.pay();
    }
}
```

Output:

```
PaymentService object created
Payment successful
```

11. Why Did the Constructor Run Immediately?

When we write:

```
ApplicationContext context =
        new ClassPathXmlApplicationContext("beans.xml");
```

Spring loads the XML file and creates the beans.

By default, Spring beans are:

```
singleton + eager
```

This means Spring creates the bean immediately when the container starts.

That is why this message prints before calling `getBean()`:

```
PaymentService object created
```

Part 2: Getting Beans from the Container

12. Get Bean by Name

```
PaymentService paymentService =
        (PaymentService) context.getBean("paymentService");
```

This works, but we need manual type casting.

---

13. Get Bean by Name and Type

Better approach:

```
PaymentService paymentService =
```

```
        context.getBean("paymentService", PaymentService.class);
```

This is safer and cleaner because it avoids manual casting.

14. Get Bean by Type

If there is only one bean of that type, we can write:

```
PaymentService paymentService =
        context.getBean(PaymentService.class);
```

This works only when Spring has exactly one matching bean of that type.

Example:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

This is fine because only one `PaymentService` bean exists.

But if we have two beans of the same type:

```
<bean id="paymentService1" class="in.strikes.PaymentService"/>
<bean id="paymentService2" class="in.strikes.PaymentService"/>
```

and we call:

```
PaymentService paymentService =
        context.getBean(PaymentService.class);
```

Spring will not randomly choose one.

It will throw:

```
NoUniqueBeanDefinitionException
```

because multiple beans match the same type.

Part 3: Bean Naming in XML

15. Bean Name Convention

Bean names usually follow Java variable naming style.

Example:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

Here, the bean name is:

```
paymentService
```

16. What If We Do Not Give an `id`?

We can write:

```
<bean class="in.strikes.PaymentService"/>
```

In this case, we have not manually given the bean a clear name.

So this will not work:

```
context.getBean("paymentService", PaymentService.class);
```

because we did not define any bean named `paymentService`.

However, this may still work:

```
PaymentService paymentService =
        context.getBean(PaymentService.class);
```

because Spring can search by type.

But again, this only works when there is exactly one matching bean of that type.

For beginner-level XML configuration, it is better to give beans a clear `id`.

---

17. What If Two Beans Have the Same `id`?

This is invalid:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

In the same XML configuration file, bean IDs should be unique.

Otherwise, Spring cannot clearly identify which bean is meant by:

```
context.getBean("paymentService");
```

---

18. Create Two Different Beans

Create another class:

```
package in.strikes;

public class OrderService {

    public OrderService() {
        System.out.println("OrderService object created");
    }

    public void placeOrder() {
        System.out.println("Order placed");
    }
}
```

Now register both beans in XML:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
<bean id="orderService" class="in.strikes.OrderService"/>
```

Use them in Java:

```
PaymentService paymentService =
        context.getBean("paymentService", PaymentService.class);

OrderService orderService =
        context.getBean("orderService", OrderService.class);

paymentService.pay();
orderService.placeOrder();
```

Output:

```
PaymentService object created
OrderService object created
Payment successful
Order placed
```

Part 4: `id`, `name`, and Alias

19. `id` Attribute

The `id` attribute gives one unique name to a bean.

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

Now we can access it using:

```
PaymentService paymentService =
        context.getBean("paymentService", PaymentService.class);
```

---

20. `name` Attribute

XML also supports the `name` attribute.

```
<bean name="paymentService" class="in.strikes.PaymentService"/>
```

This also creates a bean that can be accessed as:

```
PaymentService paymentService =
        context.getBean("paymentService", PaymentService.class);
```

At a beginner level, both of these look similar:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

```
<bean name="paymentService" class="in.strikes.PaymentService"/>
```

Both allow us to access the bean using the name `paymentService`.

---

21. Multiple Names Using `name`

The `id` attribute gives one main name.

The `name` attribute can be used for additional aliases.

Example:

```
<bean id="paymentService"
      name="payService, gatewayService, transactionService"
      class="in.strikes.PaymentService"/>
```

Now the same bean can be accessed using different names:

```
context.getBean("paymentService", PaymentService.class);
context.getBean("payService", PaymentService.class);
```

```
context.getBean("gatewayService", PaymentService.class);
context.getBean("transactionService", PaymentService.class);
```

All of these refer to the same bean.

---

22. External Alias Using `<alias>`

Sometimes a bean is already defined:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

Later, we may want to give it another name without changing the original bean definition.

We can use:

```
<alias name="paymentService" alias="payService"/>
```

Now this also works:

```
PaymentService paymentService =
        context.getBean("payService", PaymentService.class);
```

Part 5: Dependency Injection in XML

23. Two Common Ways of Dependency Injection in XML

In XML configuration, dependency injection is mainly done in two ways:

1. Constructor Injection
2. Setter Injection

XML does not directly perform field injection in the common style because XML works by calling constructors or setter methods.

> In XML configuration, Spring injects dependencies mainly through constructors or setters.

Part 6: Constructor Injection

24. Constructor Injection with Bean Reference

Suppose `OrderService` depends on `PaymentService`.

```
package in.strikes;

public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        System.out.println("OrderService constructor called");
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

XML configuration:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>

<bean id="orderService" class="in.strikes.OrderService">
    <constructor-arg ref="paymentService"/>
</bean>
```

This means:

```
Create PaymentService bean.
Create OrderService bean.
While creating OrderService, pass paymentService into its constructor.
```

The important line is:

```
<constructor-arg ref="paymentService"/>
```

Here:

Attribute	Meaning
`constructor-arg`	Pass value/dependency through constructor
`ref`	Reference to another bean in the Spring container

25. Constructor Injection with Simple Value

Suppose `PaymentService` has a constructor that takes a simple value:

```
package in.strikes;

public class PaymentService {

    private final String providerName;

    public PaymentService(String providerName) {
        this.providerName = providerName;
    }

    public void pay() {
        System.out.println("Payment successful using " + providerName);
    }
}
```

XML:

```
<bean id="paymentService" class="in.strikes.PaymentService">
    <constructor-arg value="Razorpay"/>
</bean>
```

Here:

```
value="Razorpay"
```

means Spring will pass `"Razorpay"` as a normal value, not as a bean reference.

---

26. Constructor Injection with Multiple Arguments

```
package in.strikes;

public class PaymentService {

    private final String providerName;
    private final int retryCount;

    public PaymentService(String providerName, int retryCount) {
        this.providerName = providerName;
        this.retryCount = retryCount;
    }

    public void pay() {
        System.out.println("Payment successful using " + providerName);
        System.out.println("Retry count: " + retryCount);
    }
}
```

XML:

```
<bean id="paymentService" class="in.strikes.PaymentService">
    <constructor-arg value="Razorpay"/>
    <constructor-arg value="3"/>
</bean>
```

Spring tries to match constructor arguments by order and type.

---

27. Constructor Injection by Index

To make the order clear, we can use `index`.

```
<bean id="paymentService" class="in.strikes.PaymentService">
    <constructor-arg index="0" value="Razorpay"/>
    <constructor-arg index="1" value="3"/>
</bean>
```

Here:

```
index="0" → first constructor parameter
index="1" → second constructor parameter
```

This is useful when a constructor has multiple parameters.

---

28. Constructor Injection by Name

If constructor parameter names are available, we can write:

```
<bean id="paymentService" class="in.strikes.PaymentService">
    <constructor-arg name="providerName" value="Razorpay"/>
    <constructor-arg name="retryCount" value="3"/>
</bean>
```

This is more readable.

However, constructor injection by name depends on whether Spring can discover parameter names.

For teaching examples, `index` is usually easier to demonstrate reliably.

---

29. Constructor Injection by Type

We can also specify the type:

```
<bean id="paymentService" class="in.strikes.PaymentService">
    <constructor-arg type="java.lang.String" value="Razorpay"/>
    <constructor-arg type="int" value="3"/>
</bean>
```

This helps Spring choose the correct constructor argument when matching becomes confusing.

30. Summary of Constructor Injection Options

Method	Example	Use Case
By order	`<constructor-arg value="Razorpay"/>`	Simple cases
By index	`<constructor-arg index="0" value="Razorpay"/>`	When order must be clear
By name	`<constructor-arg name="providerName" value="Razorpay"/>`	When parameter names are available
By type	`<constructor-arg type="java.lang.String" value="Razorpay"/>`	When constructor matching needs help

Part 7: Setter Injection

31. Create `OrderService` with Setter

```
package in.strikes;

public class OrderService {

    private PaymentService paymentService;

    public OrderService() {
        System.out.println("OrderService object created");
    }

    public void setPaymentService(PaymentService paymentService) {
        System.out.println("Setter called for paymentService");
        this.paymentService = paymentService;
    }

    public void placeOrder() {
        paymentService.pay();
        System.out.println("Order placed");
    }
}
```

The setter method is important:

```
public void setPaymentService(PaymentService paymentService)
```

XML setter injection works by calling this setter method.

---

32. XML Configuration for Setter Injection

```
<bean id="paymentService" class="in.strikes.PaymentService"/>

<bean id="orderService" class="in.strikes.OrderService">
    <property name="paymentService" ref="paymentService"/>
</bean>
```

The important line is:

```
<property name="paymentService" ref="paymentService"/>
```

It has two parts:

```
name="paymentService"
```

This refers to the property inside `OrderService`.

Spring looks for a setter method like:

```
setPaymentService(...)
```

Then:

```
ref="paymentService"
```

This refers to another bean in the Spring container.

So the XML means:

```
Create OrderService.
Call setPaymentService(...).
Pass the bean named paymentService into it.
```

---

33. `value` vs `ref`

In XML configuration, `value` and `ref` are different.

Attribute	Meaning
`value`	Injects a simple value like String, int, boolean
`ref`	Injects another bean from the Spring container

Example using `value`:

```
<constructor-arg value="Razorpay"/>
```

Example using `ref`:

```
<constructor-arg ref="paymentService"/>
```

Part 8: Duplicate Beans During Dependency Injection

34. Problem: Multiple Beans of Same Type

Suppose `OrderService` needs one `PaymentService`.

But the XML has two payment service beans:

```
<bean id="razorpayPaymentService" class="in.strikes.RazorpayPaymentService"/>
<bean id="stripePaymentService" class="in.strikes.StripePaymentService"/>
```

Now Spring needs to know which one should be injected into `OrderService`.

Spring should not guess randomly.

---

35. Solution 1: Explicit Wiring Using `ref`

The cleanest solution in XML is explicit wiring.

```
<bean id="razorpayPaymentService" class="in.strikes.RazorpayPaymentService"/>
<bean id="stripePaymentService" class="in.strikes.StripePaymentService"/>

<bean id="orderService" class="in.strikes.OrderService">
    <constructor-arg ref="razorpayPaymentService"/>
</bean>
```

Now there is no confusion.

This line:

```
<constructor-arg ref="razorpayPaymentService"/>
```

means:

> When creating `OrderService`, inject the bean named `razorpayPaymentService`.

Even if multiple beans are available, Spring knows exactly which one to use.

---

36. Solution 2: Using `primary="true"`

We can mark one bean as primary:

```
<bean id="razorpayPaymentService"
      class="in.strikes.RazorpayPaymentService"
      primary="true"/>

<bean id="stripePaymentService"
      class="in.strikes.StripePaymentService"/>
```

Now, when Spring needs a bean by type and multiple options exist, it will prefer the primary bean.

However, in XML projects, explicit wiring using `ref` is usually clearer than depending on `primary`.

---

37. Solution 3: Using `autowire-candidate="false"`

This is a more advanced XML option.

```
<bean id="razorpayPaymentService"
      class="in.strikes.RazorpayPaymentService"/>

<bean id="stripePaymentService"
      class="in.strikes.StripePaymentService"
      autowire-candidate="false"/>
```

This tells Spring:

> Do not consider `stripePaymentService` as a candidate for autowiring.

This can be useful in legacy XML projects, but beginners should first understand explicit wiring with `ref`.

Part 9: XML Autowiring

38. What Is XML Autowiring?

XML autowiring means:

> Instead of writing every dependency manually using `ref`, we ask Spring to find the dependency automatically.

Without autowiring:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>

<bean id="orderService" class="in.strikes.OrderService">
```

```
    <property name="paymentService" ref="paymentService"/>
</bean>
```

With XML autowiring:

```
<bean id="paymentService" class="in.strikes.PaymentService"/>

<bean id="orderService"
      class="in.strikes.OrderService"
      autowire="byName"/>
```

Spring tries to figure out the dependency automatically.

39. Common XML Autowiring Modes

Mode	Meaning
`byName`	Matches dependency using property name and bean name
`byType`	Matches dependency using type
`constructor`	Matches constructor parameters using available beans

40. `autowire="byName"`

```
<bean id="paymentService" class="in.strikes.PaymentService"/>

<bean id="orderService"
      class="in.strikes.OrderService"
      autowire="byName"/>
```

This means:

> Spring matches dependency using the property name and bean name.

If `OrderService` has this setter:

```
public void setPaymentService(PaymentService paymentService) {
    this.paymentService = paymentService;
```

```
}
```

Spring sees the property name:

```
paymentService
```

Then it searches for a bean named:

```
paymentService
```

If found, Spring injects it automatically.

---

41. `autowire="byType"`

```
<bean id="abc" class="in.strikes.PaymentService"/>

<bean id="orderService"
      class="in.strikes.OrderService"
      autowire="byType"/>
```

This means:

> Spring matches dependency using type, not name.

If `OrderService` needs a `PaymentService`, Spring searches for a bean of type `PaymentService`.

This works only when there is one clear matching bean.

If multiple beans of the same type exist, Spring cannot decide and may throw an exception.

---

42. `autowire="constructor"`

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

```
<bean id="orderService"
      class="in.strikes.OrderService"
      autowire="constructor"/>
```

This means:

> Spring uses constructor injection automatically by matching constructor parameters with available beans.

If `OrderService` has a constructor that needs `PaymentService`, Spring tries to find a matching `PaymentService` bean and pass it into the constructor.

---

43. Comparison of XML Autowiring Modes

XML Autowiring Mode	How Spring Matches Dependency	Works Best When
`byName`	Property name matches bean name	Bean names are predictable
`byType`	Property type matches bean type	Only one bean of that type exists
`constructor`	Constructor parameter type matches bean type	Constructor dependencies have clear matching beans

For beginners, it is better to first understand manual dependency injection using `ref`.

Autowiring should be treated as a convenience feature.

Part 10: Bean Scopes in XML

44. Two Important Scopes

For this video, focus mainly on two scopes:

1. `singleton`
2. `prototype`

---

45. Singleton Scope

By default, Spring beans are singleton.

```
<bean id="paymentService" class="in.strikes.PaymentService"/>
```

This means Spring creates only one object of this bean inside the container.

If we call `getBean()` multiple times:

```
PaymentService p1 =
        context.getBean("paymentService", PaymentService.class);

PaymentService p2 =
        context.getBean("paymentService", PaymentService.class);

System.out.println(p1 == p2);
```

Output:

```
true
```

Both references point to the same object.

---

46. Prototype Scope

Prototype means Spring creates a new object every time we request the bean.

```
<bean id="paymentService"
      class="in.strikes.PaymentService"
      scope="prototype"/>
```

Now:

```
PaymentService p1 =
        context.getBean("paymentService", PaymentService.class);

PaymentService p2 =
```

```
        context.getBean("paymentService", PaymentService.class);

System.out.println(p1 == p2);
```

Output:

```
PaymentService object created
PaymentService object created
false
```

Each `getBean()` call creates a new object.

---

47. Singleton Bean Depending on Prototype Bean

Suppose:

```
OrderService → singleton
PaymentService → prototype
```

XML:

```
<bean id="paymentService"
      class="in.strikes.PaymentService"
      scope="prototype"/>

<bean id="orderService"
      class="in.strikes.OrderService"
      scope="singleton">
    <constructor-arg ref="paymentService"/>
</bean>
```

Important point:

> The prototype bean is injected when the singleton bean is created.

So if `OrderService` is singleton, it receives one `PaymentService` object at creation time.

It does not automatically get a new `PaymentService` every time we call a method on `OrderService`.

This is an important concept while working with mixed scopes.

Part 11: Bean Lifecycle Methods in XML

48. Custom Init and Destroy Methods

In XML configuration, we can tell Spring which method should run:

- After bean creation
- Before bean destruction

Create `PaymentService.java`:

```
package in.strikes;

public class PaymentService {

    public PaymentService() {
        System.out.println("1. PaymentService constructor called");
    }

    public void init() {
        System.out.println("2. PaymentService init method called");
    }

    public void pay() {
        System.out.println("3. Payment successful");
    }

    public void cleanup() {
        System.out.println("4. PaymentService cleanup method called");
    }
}
```

These are normal Java methods.

They do not need annotations.

They do not need to implement any Spring interface.

The XML file tells Spring when to call them.

49. XML Configuration with `init-method` and `destroy-method`

```
<bean id="paymentService"
      class="in.strikes.PaymentService"
      init-method="init"
      destroy-method="cleanup"/>
```

Here:

Attribute	Meaning
`init-method`	Method to call after bean creation and dependency injection
`destroy-method`	Method to call before bean destruction

50. Closing the Context

To see the destroy method run, the context must be closed.

Use `ClassPathXmlApplicationContext`:

```
ClassPathXmlApplicationContext context =
        new ClassPathXmlApplicationContext("beans.xml");

PaymentService paymentService =
        context.getBean("paymentService", PaymentService.class);

paymentService.pay();

context.close();
```

Better approach using try-with-resources:

```
try (ClassPathXmlApplicationContext context =
             new ClassPathXmlApplicationContext("beans.xml")) {

    PaymentService paymentService =
            context.getBean("paymentService", PaymentService.class);
```

```
    paymentService.pay();
}
```

When the context closes, Spring calls the destroy method for singleton beans.

51. Prototype Scope with Init and Destroy

```
<bean id="paymentService"
      class="in.strikes.PaymentService"
      scope="prototype"
      init-method="init"
      destroy-method="cleanup"/>
```

For prototype beans:

- Spring creates the object.
- Spring injects dependencies.
- Spring calls the init method.
- After that, Spring hands the object to the application.

Important:

> Spring does not automatically call the destroy method for prototype beans when the context closes.

This is because Spring does not manage the complete lifecycle of prototype beans after creation.

Part 12: Collection Injection in XML

52. Injecting a List

XML can also inject collections like `List`, `Set`, and `Map`.

Example:

```
package in.strikes;

import java.util.List;

public class UserService {

    private final List<String> userNames;

    public UserService(List<String> userNames) {
        this.userNames = userNames;
    }

    public void printUsers() {
        System.out.println(userNames);
    }
}
```

XML:

```
<bean id="userService" class="in.strikes.UserService">
    <constructor-arg name="userNames">
        <list>
            <value>Aditya</value>
            <value>Rohit</value>
            <value>Rohan</value>
        </list>
    </constructor-arg>
</bean>
```

---

53. Injecting a Map

```
package in.strikes;

import java.util.Map;

public class PaymentService {

    private final Map<String, Double> userBalances;

    public PaymentService(Map<String, Double> userBalances) {
        this.userBalances = userBalances;
    }

    public void printBalances() {
```

```
        System.out.println(userBalances);
    }
}
```

XML:

```
<bean id="paymentService" class="in.strikes.PaymentService">
    <constructor-arg name="userBalances">
        <map>
            <entry key="Aditya" value="2.0"/>
            <entry key="Rohit" value="2.5"/>
            <entry key="Rohan" value="3.0"/>
        </map>
    </constructor-arg>
</bean>
```

This shows that XML can handle not only simple beans, but also more complex values and collections.

Part 13: Splitting XML Files

54. Why Split XML Files?

In real legacy Spring projects, one XML file can become very large.

Instead of keeping everything inside one `beans.xml`, we can split configuration into multiple XML files.

Example:

```
src/main/resources
 ├── applicationContext.xml
 ├── orderContext.xml
 └── userContext.xml
```

---

55. Example: `orderContext.xml`

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="paymentService" class="in.strikes.PaymentService"/>

    <bean id="orderService" class="in.strikes.OrderService">
        <constructor-arg ref="paymentService"/>
    </bean>

</beans>
```

56. Example: `userContext.xml`

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="userService" class="in.strikes.UserService"/>

</beans>
```

57. Main File: `applicationContext.xml`

Now create a main XML file that imports other XML files.

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
    <import resource="orderContext.xml"/>
    <import resource="userContext.xml"/>

</beans>
```

Now in Java, we only load the main file:

```
ClassPathXmlApplicationContext context =
        new ClassPathXmlApplicationContext("applicationContext.xml");
```

Spring will load `applicationContext.xml`, and that file will import the other XML files.

Part 14: XML and Annotation Configuration Together

58. Can XML and Annotations Work Together?

Yes.

XML and annotations can work together because both are just different ways of giving configuration metadata to Spring.

For example, an old project may have:

- Some beans defined in XML
- Some beans detected using annotations
- Some beans created using Java config

Ultimately, Spring converts all of them into bean definitions.

---

59. Component Scanning from XML

In XML, we can enable annotation scanning using:

```
<context:component-scan base-package="in.strikes"/>
```

For this, the XML file needs the context namespace:

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd
           http://www.springframework.org/schema/context
           http://www.springframework.org/schema/context/spring-context.xsd">

    <context:component-scan base-package="in.strikes"/>

</beans>
```

This tells Spring:

> Scan the package `in.strikes` and detect annotation-based components like `@Component`, `@Service`, and `@Repository`.

---

60. Should Beginners Mix XML and Annotations?

For learning, it is better to understand both separately first.

Mixing XML and annotations too early can become confusing because:

- XML beans can depend on annotation-based beans.
- Annotation-based beans can depend on XML beans.
- Component scanning and manual bean registration can create duplicate beans.
- Debugging becomes harder for beginners.

So the best teaching approach is:

1. First understand pure annotation-based configuration.
2. Then understand pure XML-based configuration.
3. Then briefly explain that both can work together in legacy projects.

Final Summary

Key Takeaways

- XML configuration is an older but important way of configuring Spring applications.
- XML files provide configuration metadata to Spring.
- Spring converts XML configuration into `BeanDefinition` objects.
- `<bean>` tells Spring to create and manage an object.
- `id` gives the bean a name inside the container.
- `class` tells Spring which class object should be created.
- Dependencies can be injected using constructor injection or setter injection.
- `value` is used for simple values.
- `ref` is used for another Spring bean.
- If multiple beans of the same type exist, explicit wiring using `ref` is the clearest solution.
- XML also supports autowiring using `byName`, `byType`, and `constructor`.
- The default bean scope is singleton.
- Prototype scope creates a new object every time `getBean()` is called.
- `init-method` and `destroy-method` allow lifecycle callbacks in XML.
- Spring does not automatically destroy prototype beans.
- Large XML configurations can be split into multiple files using `<import>`.
- XML and annotations can work together, but beginners should first understand them separately.